CHRISTOPHER NEWPORT UNIVERSITY · SCHOOL OF ENGINEERING & COMPUTING

Training Neural Networks

Learning Rate Schedulers

How changing the learning rate during training improves the model

Keith Perkins · School of Engineering & Computing

START HERE

The learning rate is the step size

GRADIENT DESCENT, ONE STEP

$$w \leftarrow w - \eta \cdot dL/dw$$

w: a weight in the model

dL/dw: which way is downhill (the gradient)

η (eta): the learning rate: how far to step

Too big and you overshoot. Too small and training crawls.

EXAMPLE: MINIMIZE $L(W) = W^2$, STARTING AT $W = 3$

The gradient is 2w, so each step is $w \leftarrow w - \eta \cdot 2w$

$\eta = 0.01$	3 → 2.94 → 2.88 → 2.82	Too slow
$\eta = 0.1$	**3 → 2.4 → 1.92 → 1.54**	**Good**
$\eta = 0.9$	3 → −2.4 → 1.92 → −1.54	Bounces back and forth
$\eta = 1.1$	3 → −3.6 → 4.32 → −5.18	Diverges!

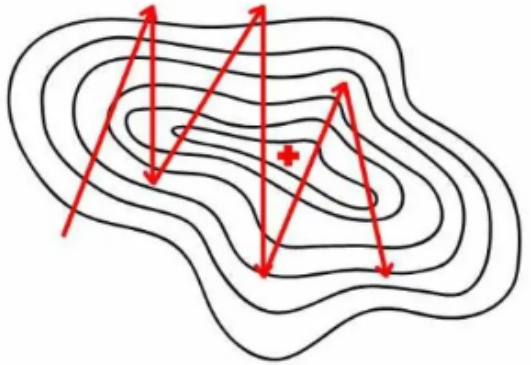
THE PROBLEM

Why change the learning rate during training?

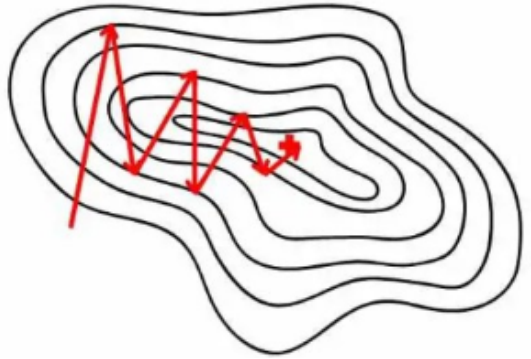
A fixed learning rate forces one compromise for the whole run: large enough to make progress early, small enough to settle at the end. Usually it's neither.

A scheduler changes the learning rate as training goes on: big steps early to move fast, small steps late to settle into a good minimum.

Result: faster convergence, more stable training, and often better generalization.



Step too large: oscillating



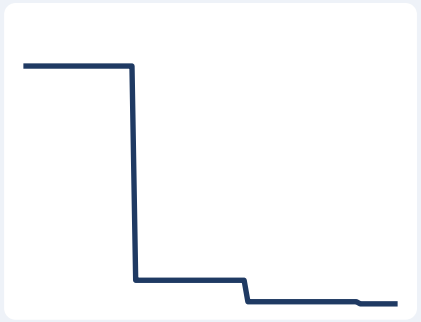
Steps shrink: converging

THE MENU

Common learning rate schedulers

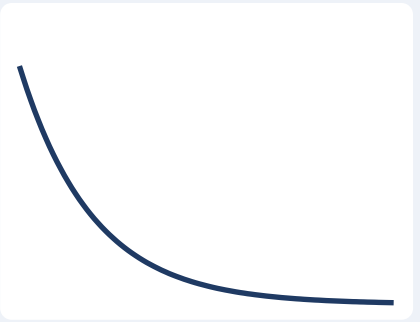
Step decay

Cut the rate by a fixed factor every few epochs (here ×0.1 every 30).



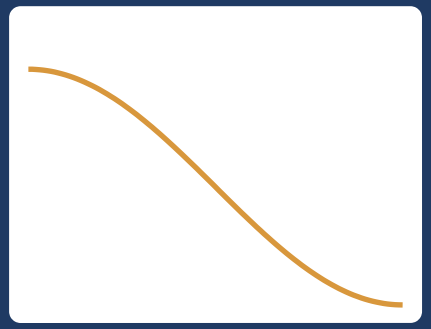
Exponential decay

Multiply by a constant each epoch (here ×0.95) for a smooth drop.



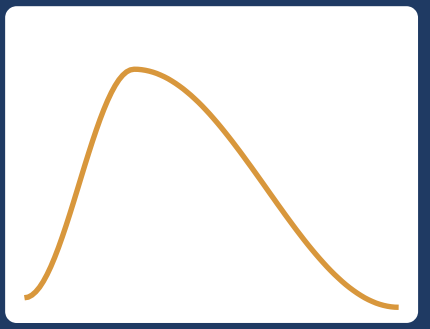
Cosine annealing

Follow half a cosine wave from high to low, with no abrupt jumps.



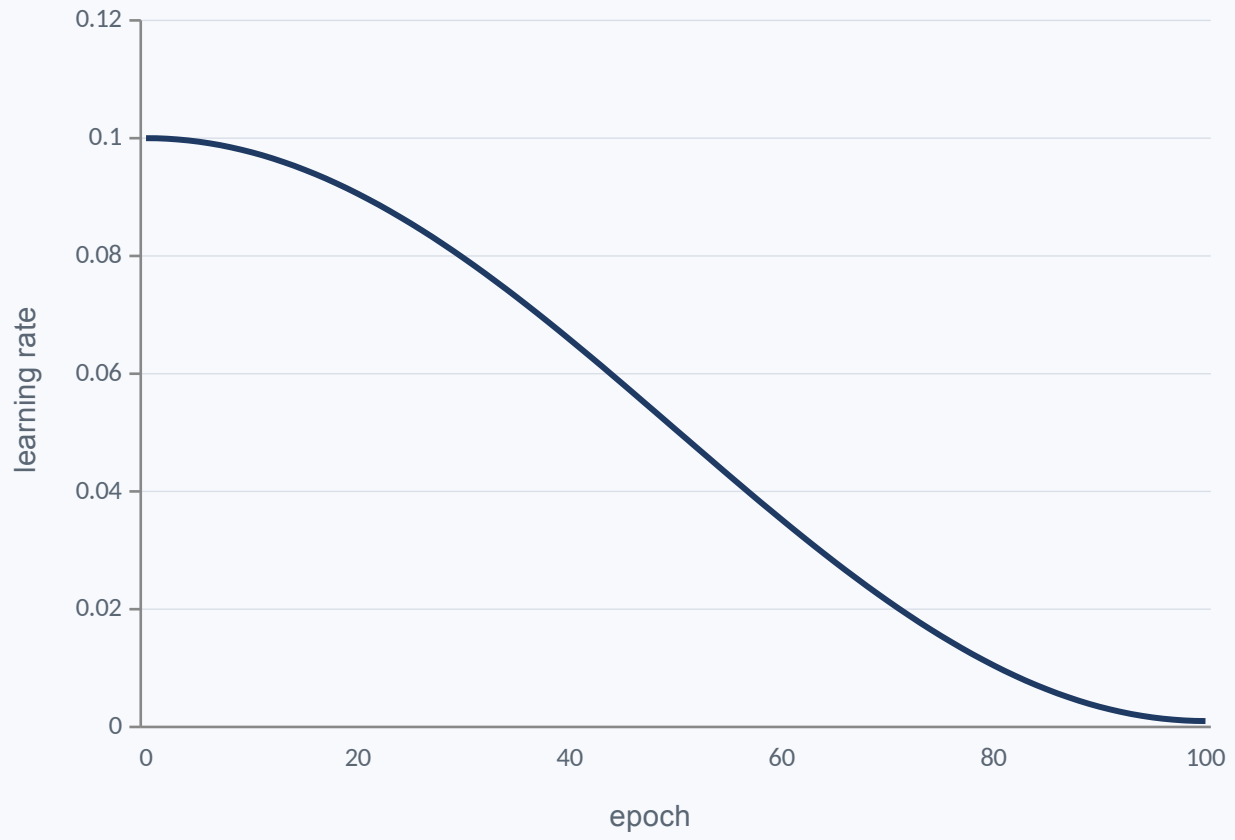
Cyclic / One cycle

Raise the rate, then lower it, once or repeatedly, to explore before settling.



We'll cover the two on the bottom: cosine annealing (with restarts) and one cycle.

Cosine annealing: the intuition



Smooth cosine curve

Starts at a high learning rate and glides down along a cosine curve, with no sudden drops.

Borrowed from annealing

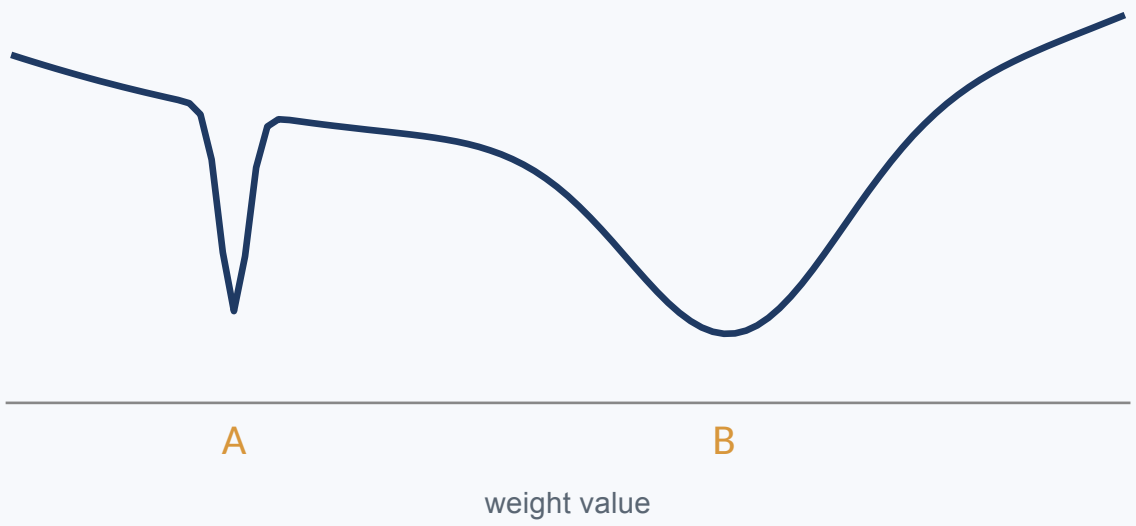
Named after simulated annealing, which "cools" a search over time so it explores less and less.

Explore, then refine

Big steps early explore the loss surface; small steps late fine-tune into a minimum.

Why should big steps lead somewhere good?

AN ILLUSTRATIVE 1-D LOSS SURFACE



A · sharp, narrow minimum
A big, noisy step bounces right out, so the early phase rarely stays here.

B · wide, flat basin
Big steps can stay inside it; small steps later settle to the bottom. Wide basins tend to generalize better.

1 Noisy steps can't settle

Each step uses a mini-batch, so it is noisy. At a fixed rate the weights keep bouncing in a region near the minimum, and bigger steps make a bigger region. Shrinking the rate shrinks the bounce.

2 Big steps act as a filter

They skip over sharp minima like A and only stay in wide basins like B.

3 It works in practice

Loss levels off while the rate is high, then drops sharply when the rate comes down.

No guarantee: it's a well-tested heuristic, not a proof.

The formula, with numbers

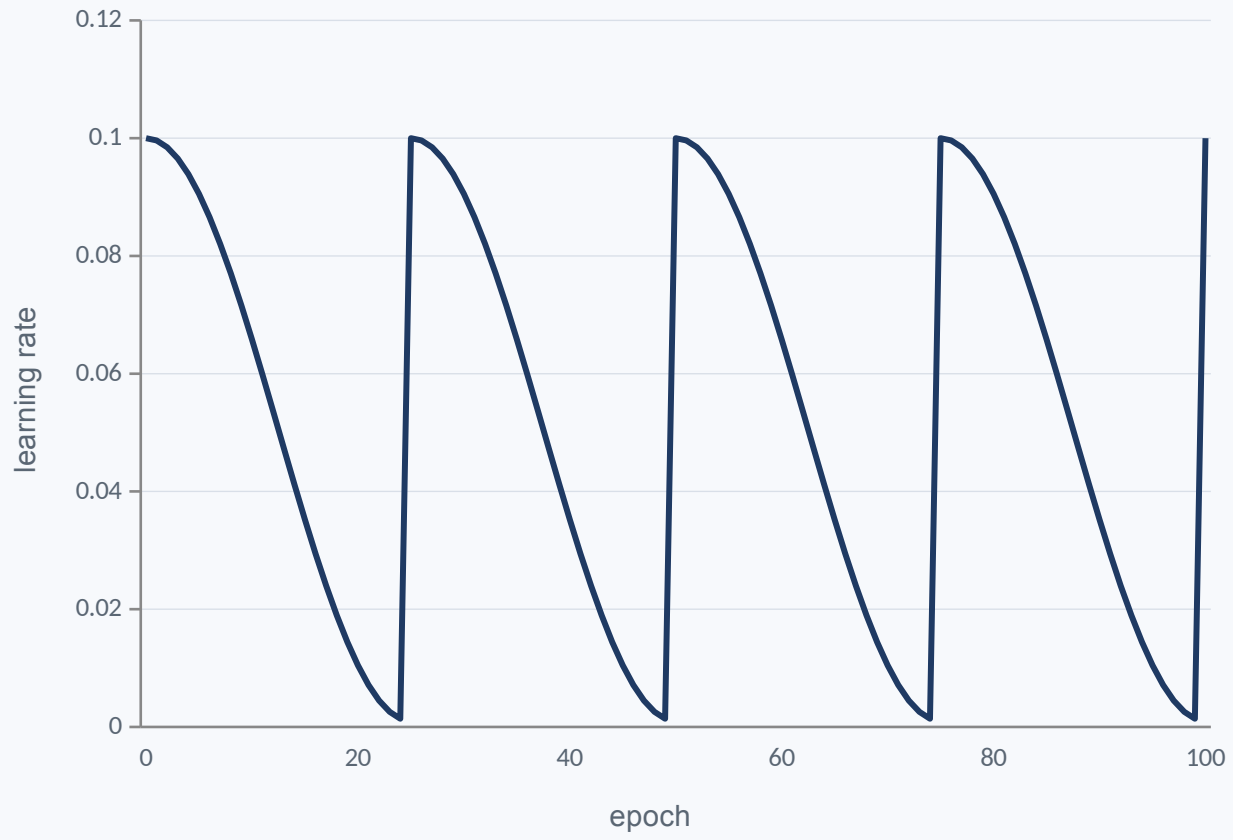
$$\eta_t = \eta_{min} + \tfrac{1}{2}(\eta_{max} - \eta_{min}) \cdot (1 + \cos(\pi \cdot t / T_{max}))$$

`η_max`	Starting (highest) learning rate: the lr you give the optimizer
`η_min`	Lowest learning rate, reached at the end
`T_max`	How many epochs (or steps) the decay takes

EXAMPLE: H_MAX = 0.1, H_MIN = 0.001, T_MAX = 100

t	cos(π·t/100)	η_t
0	1.00	**0.1000**
25	0.71	**0.0855**
50	0.00	**0.0505**
75	−0.71	**0.0155**
100	−1.00	**0.0010**

Cosine annealing with warm restarts (SGDR)



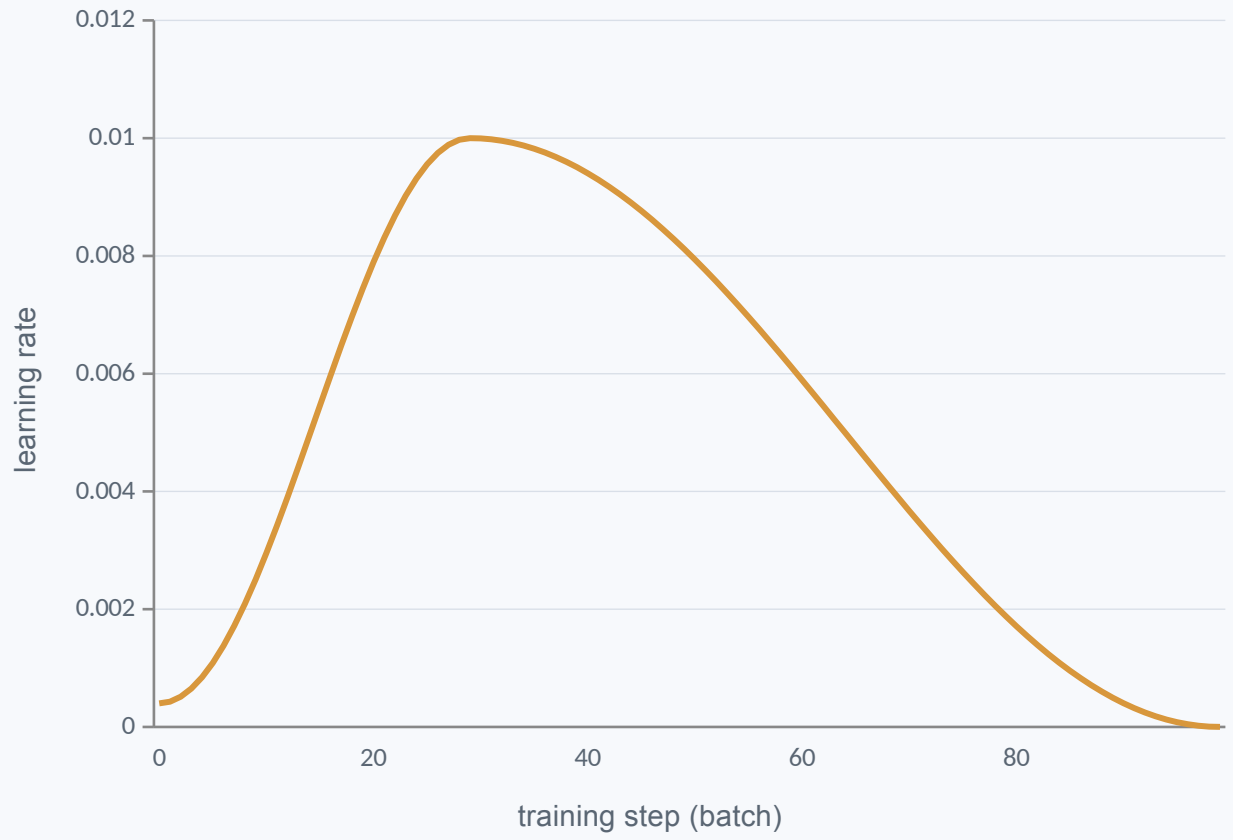
From "SGDR: Stochastic Gradient Descent with Warm Restarts" (Loshchilov & Hutter, 2016).

Every T_0 epochs the learning rate jumps back to η_max and decays again. Each restart shakes the model out of where it settled so it can explore other regions of the loss surface.

PYTORCH

```
CosineAnnealingWarmRestarts(
    optimizer, T_0=25, T_mult=1)
# T_mult=2 doubles each cycle
```

One cycle: up, then down



From "Super-Convergence" (Smith & Topin, 2018).

- Warm up from max_lr ÷ 25 to max_lr
- Then anneal to far below where it started
- pct_start (default 0.3) = fraction of training spent going up

PYTORCH · STEPS EVERY BATCH

```
OneCycleLR(optimizer, max_lr=0.01,
    epochs=E,
steps_per_epoch=len(loader))
```

IN PRACTICE

Adding a scheduler to a PyTorch training loop

PYTHON · PYTORCH

```
optimizer = torch.optim.SGD(model.parameters(),
                            lr=0.1, momentum=0.9)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
                optimizer, T_max=num_epochs, eta_min=1e-3)

for epoch in range(num_epochs):
    for x, y in train_loader:
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
    scheduler.step()            # once per epoch
    print(epoch, scheduler.get_last_lr())
```

COMMON MISTAKES

Wrong order

Call scheduler.step() after optimizer.step(), never before.

Wrong frequency

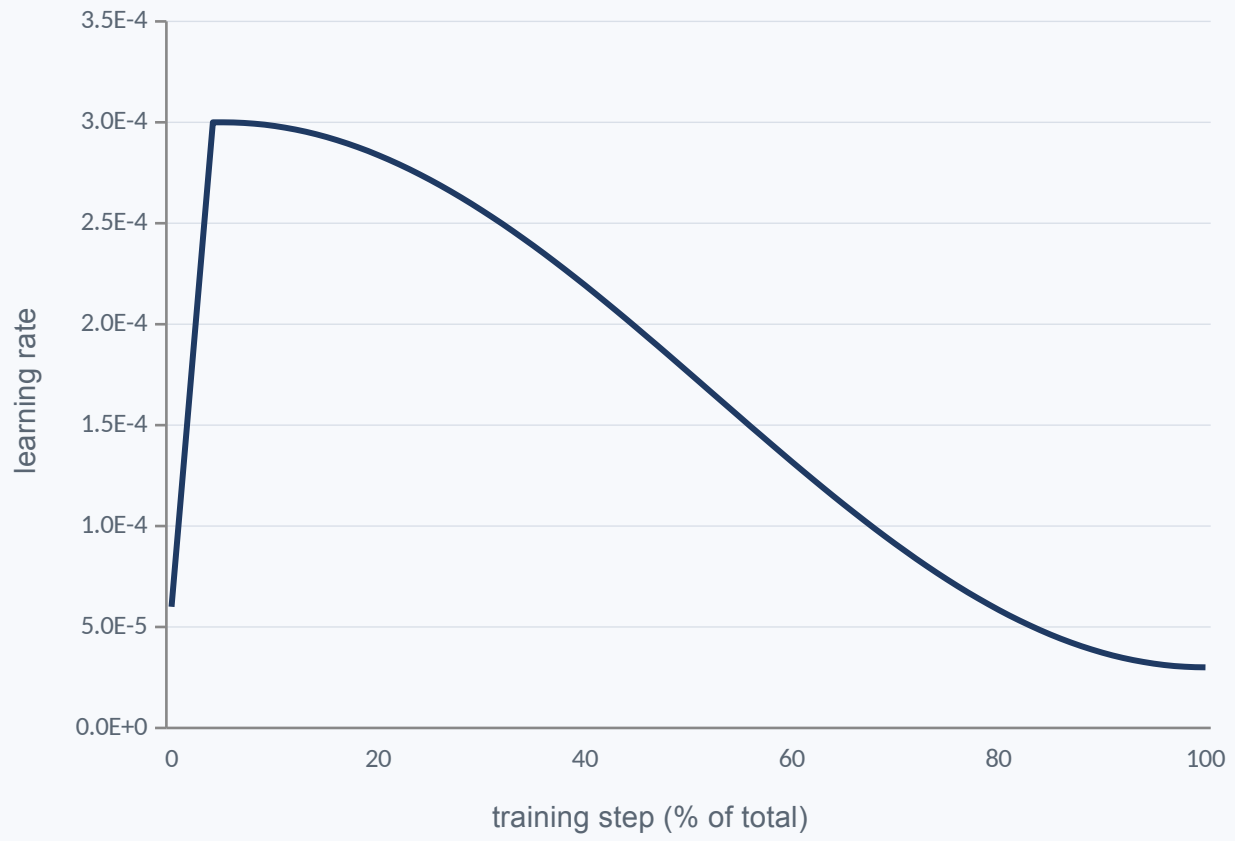
Cosine and step decay: once per epoch. OneCycleLR: once per batch.

Forgetting η_max

The lr you give the optimizer is the starting (maximum) rate.

WHY WE CARE

LLMs use warmup, then decay



THE STANDARD RECIPE

1 · Warmup: raise the rate from near 0 over the first small fraction of steps (often 1–10%)

2 · Decay: cosine down to about 10% of the peak

3 · No warm restarts: one warmup, one long decay

MENTAL MODEL

Warmup: learn to walk before running.

Decay: slow down so you don't overshoot the minimum.

A COMMON TRAP

Don't cut a planned schedule short

Why: cosine and one-cycle schedules plan the whole run in advance. Stop early, and you stop while the learning rate is still high, before the low-rate phase where the model settles.

1

Finish the schedule

Let the scheduler complete its planned epochs or steps.

2

Keep the best checkpoint

Save the model whenever validation improves, and use that one at the end.

3

Shorten the plan instead

Worried about overfitting? Lower the total epochs (or T_max) and rerun.

Exception: ReduceLROnPlateau watches validation loss and lowers the rate when progress stalls, so it pairs naturally with early stopping.

SUMMARY

Key takeaways

Learning rate = step size

Too big overshoots, too small crawls. A scheduler changes it as training goes.

Cosine annealing

Smooth decay from η_max to η_min; warm restarts jump back up to explore again.

One cycle

Ramp up, then down. Step it every batch, not every epoch.

LLMs: warmup + decay

A short warmup, then cosine decay. Finish the schedule and keep the best checkpoint.

Schedulers are a cheap upgrade: one extra line of code, and often faster, more stable training.